F1 StrategyIQ

Complete Phase-Wise Implementation Roadmap

Data-Driven Race Strategy Optimization & Dynamic Pit-Wall Decision Support System

This is the master roadmap for building the project after the Streamlit frontend. It defines every phase, implementation task, inputs, outputs, technologies, validation and final integration.

1. Overall Architecture

Historical F1 Data → Data Collection → PostgreSQL → Cleaning & Feature Engineering → EDA → ML → Race Simulation → Monte Carlo → Strategy Optimization → Dynamic Pit-Wall → FastAPI → Streamlit

Core principle: Analyze → Predict → Simulate → Optimize → Decide → Re-evaluate.

2. Phase Overview

Phase	Name	Main Deliverable
0	Frontend	Interactive Streamlit dashboard — COMPLETED
1	Data Collection	Raw F1 datasets + reproducible ingestion
2	Database	PostgreSQL schema + populated database
3	Cleaning & Features	Clean ML-ready feature dataset
4	EDA & Statistics	Insights, statistical analysis and charts
5	Machine Learning	Lap-time, tyre and pit-decision models
6	Race Simulation	Candidate strategy simulator
7	Monte Carlo	Outcome probabilities and uncertainty
8	Strategy Optimization	Ranked/recommended strategies
9	Dynamic Pit-Wall	Race-state decision engine
10	FastAPI	Backend API
11	Integration	Real frontend + backend system
12	Testing	Model and end-to-end validation
13	Deployment	Final application, report and demo

3. PHASE 0 — Frontend (COMPLETED)

Built with Streamlit and Plotly. Pages: Overview, Race Analysis, Tyre Intelligence, Strategy Optimizer, Dynamic Pit-Wall and What-If Simulator. The current frontend uses mock data; later the mock functions are replaced with FastAPI calls.

4. PHASE 1 — Data Collection

Goal: obtain reliable public F1 data. Recommended scope: 6–8 diverse circuits across multiple recent seasons, e.g. 2021–2025, while accounting for regulation changes.

Sources: FastF1 for detailed timing, laps, telemetry, tyres, weather and track status; Jolpica F1 API for structured historical results, drivers, constructors, standings, laps and pit stops; optional public/Kaggle datasets for cross-checking.

Implementation: create ingestion scripts → standardize IDs/names → save raw data unchanged → record source/session metadata → validate completeness.

Output: raw datasets + data dictionary + reproducible ingestion pipeline.

5. PHASE 2 — PostgreSQL Database

Table	Purpose
races	Season, round, race, circuit and date
circuits	Circuit identity, location and characteristics
drivers	Driver identity and metadata

Table	Purpose
constructors	Team/constructor information
results	Grid, finish, points and status
laps	Lap number, time, position and sectors
tyre_stints	Compound, stint, start/end lap and tyre age
pit_stops	Pit lap and pit duration
weather	Air/track temperature, humidity, rainfall and wind
features	Optional derived ML-ready features

Implementation: schema → primary/foreign keys → ingestion-to-DB script → constraints/indexes → row-count and relationship validation.

> **Important limitation:** exact fuel load, setup, engine modes and proprietary tyre-state information are not public. Do not invent them.

6. PHASE 3 — Cleaning & Feature Engineering

Clean missing/invalid values, duplicates, tyre labels, red flags, Safety Car/VSC periods and outliers. Create tyre_age, rolling_lap_time, recent_pace, gap_ahead, gap_behind, laps_remaining, degradation_rate, pit_loss, temperatures, rainfall, safety_car, driver, constructor, circuit and season features.

> **Critical:** no future information can enter features at a decision point. This prevents data leakage.

Output: clean dataset + ML feature dataset + feature dictionary.

7. PHASE 4 — EDA & Statistical Analysis

Analyze tyre age vs pace, compound performance, driver consistency, position changes, gaps, pit loss, circuit behavior and weather effects. Produce correlation analysis where appropriate and distributions/trend analysis.

> **Visuals:** degradation curves, lap-time distributions, driver comparisons, position charts, pit-stop distributions, compound comparisons, correlation heatmap and circuit comparisons.

8. PHASE 5 — Machine Learning

Model A — Lap-Time Regression: Inputs: driver, constructor, circuit, compound, tyre age, lap, recent pace, weather and race-state variables. Compare Linear Regression, Random Forest and XGBoost. Output: predicted lap time. Metrics: MAE, RMSE, R^2.

Model B — Tyre Degradation: Inputs: compound, tyre age, circuit, driver, pace and conditions. Outputs: degradation rate, expected pace loss and projected tyre window.

Model C — Pit Decision Classification: Inputs: lap, position, tyre age, gaps, pace, degradation, remaining laps, pit loss and Safety Car/weather. Outputs: PIT NOW / STAY OUT, optionally PIT NEXT/WINDOW, plus confidence. Metrics: precision, recall, F1 and ROC-AUC.

Validation: use race-level/chronological splits rather than random lap-level splitting to reduce leakage.

9. PHASE 6 — Race Simulation

Generate feasible tyre/pit strategies such as M→H, M→M→H and S→M→H. For every strategy, project lap performance, degradation, pit loss, remaining laps, position and traffic assumptions.

Outputs: predicted race time, finishing position and strategy trajectory.

10. PHASE 7 — Monte Carlo Simulation

Repeat strategy simulations many times while varying defensible uncertainties such as degradation, model error, traffic and Safety Car/weather scenarios. Use data-driven distributions where possible and document assumptions.

Outputs: win probability, podium probability, expected position, finishing-position distribution and risk/variability.

Visuals: finishing-position distribution, probability bars and strategy outcome distributions.

11. PHASE 8 — Strategy Optimization

Compare simulated strategies using expected race time plus finishing position, podium/win probability and risk. Document objective weights and perform sensitivity testing.

Output: ranked strategies + preferred strategy + explanation. Validate against actual historical strategies and simple rule-based baselines; do not claim absolute optimality.

12. PHASE 9 — Dynamic Pit-Wall Engine

Start with an initial strategy and re-evaluate meaningful race-state updates. Do not attempt millisecond-level professional live timing. Use lap-by-lap historical replay plus events such as Safety Car/VSC, competitor pit stops, major gap changes, degradation thresholds and weather changes.

Flow: current state → features → ML prediction → strategy alternatives → simulation → optimization → PIT NOW/STAY OUT → next state → repeat.

Output: action, confidence, recommended compound/pit window, projected outcome and reasons.

13. PHASE 10 — FastAPI Backend

FastAPI connects Streamlit with PostgreSQL, ML models and the strategy engine. Train models offline and save them with Joblib/Pickle; the API loads them rather than retraining on each request.

Endpoint	Purpose
GET /api/races	List available races
GET /api/races/{race_id}	Race details/state
GET /api/drivers/{driver_id}/performance	Driver analysis
POST /api/predict/lap-time	Lap-time prediction
POST /api/predict/tyre	Tyre degradation prediction
POST /api/strategy/optimize	Simulation + strategy optimization
POST /api/pitwall/decision	Dynamic Pit-Wall recommendation

Implementation order: Pydantic schemas → DB service → model service → strategy service → endpoints → validation/error handling → API tests.

14. PHASE 11 — Frontend + Backend Integration

> Replace mock frontend functions with API calls. Final flow: **Streamlit selection → FastAPI → PostgreSQL → feature pipeline → ML → simulation/optimization → structured response → Plotly/cards/tables/timelines.**

The API should return structured values such as predicted_lap_time, degradation_rate, recommended_action, confidence, recommended_compound, expected_finish, podium_probability, win_probability and risk_level.

15. PHASE 12 — Testing & Validation

Area	Tests
Data	Missing values, duplicates, ranges, foreign keys, row counts
Regression	MAE, RMSE, R^2; model comparison
Classification	Precision, recall, F1, ROC-AUC; class balance
Leakage	Chronological/race-level split and feature audit
Simulation	Historical race replay and sanity checks
Strategy	Compare recommendations with actual/rule-based strategies
API	Schema validation, invalid input, endpoint tests
Frontend	Selections, charts, replay and what-if controls
End-to-end	User input → API → DB/model → correct visual output

> **Success criterion:** repeatable, explainable results with uncertainty clearly communicated; predictions are not guarantees.

16. PHASE 13 — Deployment & Documentation

Deploy only after local end-to-end testing. Possible services: Supabase PostgreSQL, Render/Railway for FastAPI and Streamlit Community Cloud for the dashboard.

Documentation should include architecture diagram, ER diagram, data dictionary, ML methodology, metrics, limitations, screenshots, API documentation, setup instructions and user guide.

17. Three-Month Schedule

Weeks	Focus	Milestone
1–2	Data collection	Raw data + documentation
3	Database	PostgreSQL schema + ingestion
4	Cleaning/features	ML-ready dataset
5	EDA/statistics	Insights + visualizations
6	Lap-time ML	Best regression model

Weeks	Focus	Milestone
7	Tyre ML	Degradation model
8	Pit classifier	Decision model
9	Simulator	Strategy simulation
10	Monte Carlo + optimizer	Probabilities + strategy ranking
11	Pit-Wall + FastAPI	Decision engine + API
12	Integration/testing	Complete application + report/demo

18. Suggested Team Split

Member 1: data collection, cleaning and PostgreSQL. **Member 2:** ML models and evaluation. **Member 3:** simulation, Monte Carlo, optimizer and Pit-Wall. **Member 4:** FastAPI, integration and testing. Adjust according to team size; everyone should understand the full architecture for the viva.

19. What NOT to Build

Avoid unless time remains: full reinforcement learning, computer vision, millisecond live streaming, full physics simulation, deep learning everywhere, authentication and a complex React frontend. They add scope without being necessary for the core 40-mark objective.

20. Final Demonstration Flow

Select GP/driver → inspect historical race and tyre visuals → select current race state → run predictions → compare strategies → run Monte Carlo → show recommended strategy → replay race lap-by-lap → demonstrate PIT NOW/STAY OUT changing after an event → run a what-if scenario → explain metrics and limitations.

21. Final End-to-End Definition

Historical data provides the foundation. **ML** predicts uncertain race behavior. **Simulation** explores possible futures. **Optimization** chooses among strategies. **Dynamic Pit-Wall** turns changing race state into an action. **FastAPI** exposes the intelligence. **Streamlit** turns the results into an interactive visual application.

MASTER PRINCIPLE: Analyze → Predict → Simulate → Optimize → Decide → Re-evaluate.